MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Expert offloading enables single-GPU inference for mixture-of-experts (MoE) models. Repeated HDD reads make expert residency a source of inference latency. We present MECC, the MoE Expert Cache Compiler, for profile-guided placement and joint HBM/CPU caching. Its compiler allocates slots by layer and emits the expert IDs used for initialization. The runtime promotes router-selected experts and retains displaced GPU weights in the host cache. The delivery path orders HDD reads and H2D copies before GPU consumption. On one A100, MECC gives 1.77× decode speedup over streaming on FP16 Mixtral-8x7B. First-token speedup reaches 2.86× over a Fiddler-style adapter. The evaluated Mixtral perplexity is 3.427. Joint retention reduces HDD demand by 73.9% relative to independent caches.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models scale parameter capacity through sparse expert activation [1]–[4]. For example, Mixtral selects two of eight experts in each layer. A single-GPU deployment can use high-bandwidth memory (HBM), host memory, and a hard disk drive (HDD) to hold that pool. The selected expert's location determines its delivery path. HBM residents are ready for GPU execution; host residents require a host-to-device (H2D) copy; cold experts require an HDD read followed by H2D. Layer dependencies expose these reads on the inference critical path. Prefill can reuse fetched weights across prompt rows, and decoding can reuse them across output tokens. On shared servers, a serving task manages its expert allocation alongside other jobs and model instances. We study how to distribute that allocation across layers and retain weights for recurring requests. All MoE layers share the HBM and host allowances. Increasing one layer's allocation leaves fewer slots for others. The value of another slot depends on the demand already covered in that layer. Placement therefore couples the layers' local request distributions to a shared storage allocation.

Expert parallelism places experts across GPUs [5]–[7]. Single-GPU systems combine expert paging and caching [8]–[11] with CPU/GPU execution [12]–[14]. MoE-CORE combines layer-wise allocation, profile initialization, and online replacement with cross-layer prefetching [15]. It also supports substitution for eligible expert misses. SSD-LLaMA retains GPU victims in RAM and pipelines storage delivery alongside lossless compression and CPU/GPU work partitioning [16]. Edge FlashMoE studies cache replacement for storage-backed expert access [17]. Within such a hierarchy, initial placement and later admission affect different requests. A profile determines which weights are available when serving begins. GPU turnover then determines which of those weights survive on the host. Promoting a CPU expert vacates a host position just as an HBM victim loses its GPU position. Using that position for the victim preserves its place in the reuse order. Figure 1 compares this joint rule with independent HBM/CPU caches on an A100. Joint retention gives 4.46× decode speedup over independent replacement and reduces HDD demand by 73.9%. MECC also gives 1.77× speedup over streaming. These comparisons motivate retaining reusable expert weights across both memory tiers throughout serving.

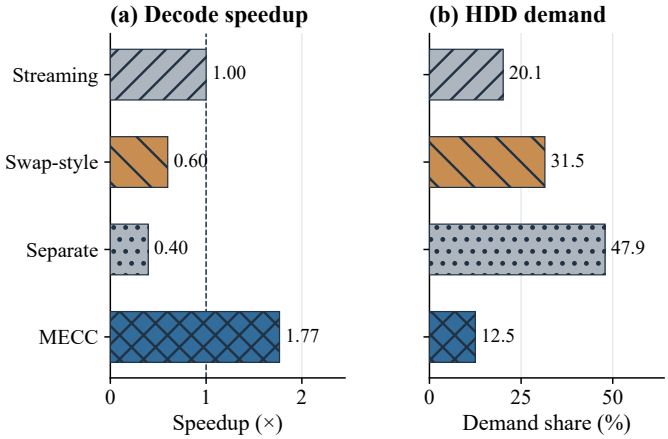


Fig. 1. Mixtral-8x7B on one A100. Decode speedup is relative to streaming; Separate uses independent HBM/CPU caches. Swap-style is the SwapMoE-style adapter. Higher speedup and lower HDD demand are better.

We present MECC, the MoE Expert Cache Compiler, to connect these two stages. Compilation ranks experts within each layer and allocates slots by marginal covered demand. The resulting plan specifies both per-layer capacities and the IDs loaded into HBM and CPU. The runtime maintains a joint recency order across those tiers. A CPU hit promotes the selected expert and demotes the GPU victim; a cold admission can also evict a host resident. A host mirror supplies the weights of each demoted GPU resident. Host admission reuses this copy through a metadata update. Stream tracking keeps tensor storage live through kernel execution. Runtime requests update membership within the compiled capacities. Weight delivery follows the selected experts' read and copy dependencies before GPU execution. Prefill submits cold reads for the active expert batch, while decode loads selected experts on demand. The compiler and runtime address each weight block by its layer/expert ID. This key connects a profiled rank to its initial location, later cache updates, and FP16 execution.

This paper makes three contributions.

(1) **Compiling initial residency.** We map layer profiles and expert allowances to per-layer capacities and ranked expert IDs. A marginal-coverage scan determines how each additional slot extends the initial resident set.

(2) **Updating cross-tier residency.** We connect the compiled state to joint promotion and victim demotion. Host retention preserves GPU evictions for later reuse, while read/copy ordering prepares selected weights for consumption.

(3) **Evaluating weight delivery.** We compare placement, retention, and scheduling through decode latency, first-token time, source-tier demand, and sample perplexity. Workload and capacity sweeps examine calibration, task transfer, prompt length, batch size, and host allocation. Deployment comparisons report latency and accelerator occupancy.

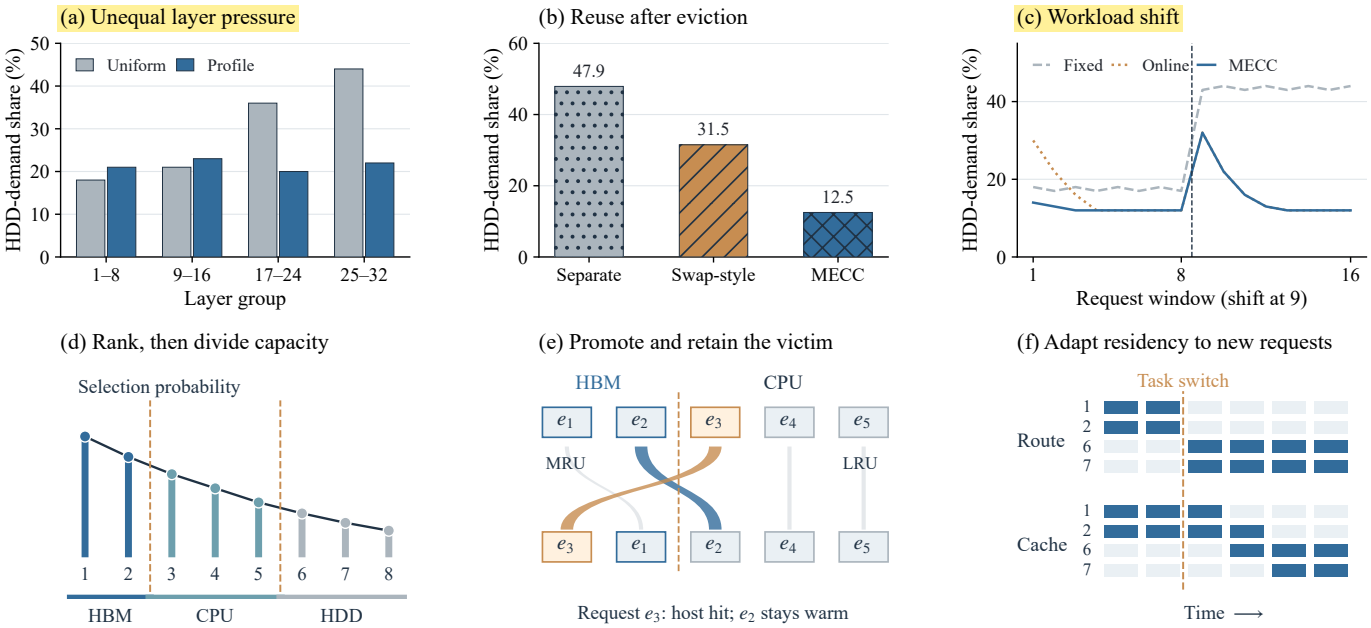


Fig. 2. Expert-cache behavior and MECC's three design choices. (a–c) compare layer pressure, cross-tier reuse, and workload change; (b) counts A100 demand across all phases. (d–f) show layer-specific capacity cuts, joint victim retention, and online membership updates.

II. Background

A. Sparse Execution, Full Weight Capacity

An MoE layer routes each token to a subset of experts and combines their outputs [1], [2], [18]. Mixtral and the DeepSeek family separate active computation from the size of the stored expert pool [3], [4], [19]–[21]. An expert belongs to one layer; equal indices at different layers identify different weights. For input x, let $G(x)$ be the selected set and $g_e(x)$ its normalized gate weights. The layer computes

$$y(x) = \sum_{e \in G(x)} g_e(x) E_e(x). \tag{1}$$

Each gate produces expert requests as a token traverses the model. Decode revisits the layers for each output token. An expert request triggers a residency lookup, weight delivery, and execution on its assigned token rows. The gate weights then combine the expert outputs.

B. Expert Residency on One GPU

Expert parallelism places experts on GPU owners and dispatches token activations to them [5]–[7], [22], [23]. Single-GPU offloading delivers selected weights to the execution device. Experts share HBM with dense weights, activations, and the KV cache [24], [25]. The host cache supplies H2D copies; HDD supplies weights absent from both resident tiers. Heterogeneous inference systems combine these paths with CPU/GPU scheduling [26]–[31]. In prefill, several prompt rows can share a fetched expert. Decode advances one token per sequence, exposing fewer rows to each active expert. The amount of ready computation therefore changes between the two phases. Other systems change transfer size through precision selection [32], [33], expert segmentation [34], [35], or staging and compression [36], [37].

Calibration counts determine expert ranking; runtime accesses determine recency. Within one layer, sequences $[a, a, b, b]$ and $[b, b, a, a]$ have equal counts and different last-access orders. MECC initializes from the ranked profile and replaces residents by recency.

III. Motivation and Insights

A. Allocating Slots by Marginal Demand

Expert demand varies across layers. Some layers concentrate requests in a few ranks; others spread requests across more experts. Figure 2(a,d) illustrates the resulting allocation choice. Its profile-guided plan shifts slots toward the layer groups with more cold demand. The compiler makes this choice by comparing each layer's next uncovered rank. Allocating a slot exposes that layer's following rank as a new candidate. This process distributes a shared allowance across layers and retains the IDs that fill each allocation. Routing locality across models has also been studied independently [38].

B. Preserving Reuse after GPU Eviction

A GPU victim is also a candidate for host admission. Figure 2(b,e) shows a CPU promotion vacating a host position as the GPU selects its victim. Joint retention admits that victim into the vacated position, carrying recency across the tier boundary. Starting from the same allocation and expert order, joint retention reduces HDD demand from 47.9% to 12.5%. A demoted expert enters CPU MRU and remains available until subsequent admissions evict it from CPU LRU. A later selection promotes this host copy through H2D, turning the earlier GPU eviction into a host-cache hit.

C. Updating Residency beyond Calibration

Calibration and serving inputs can select different expert sets. In Figure 2(c,f), a task switch moves requests toward a new expert set. MECC initializes the resident set from calibration and then admits experts requested during serving. Each admission updates recency and brings the cache closer to the active task. Repeated requests keep the new experts near MRU; their unused predecessors advance toward eviction. The profile supplies an initial hot set, and request-driven replacement tracks its evolution. The compiled capacities assign each layer the space used for this adaptation.

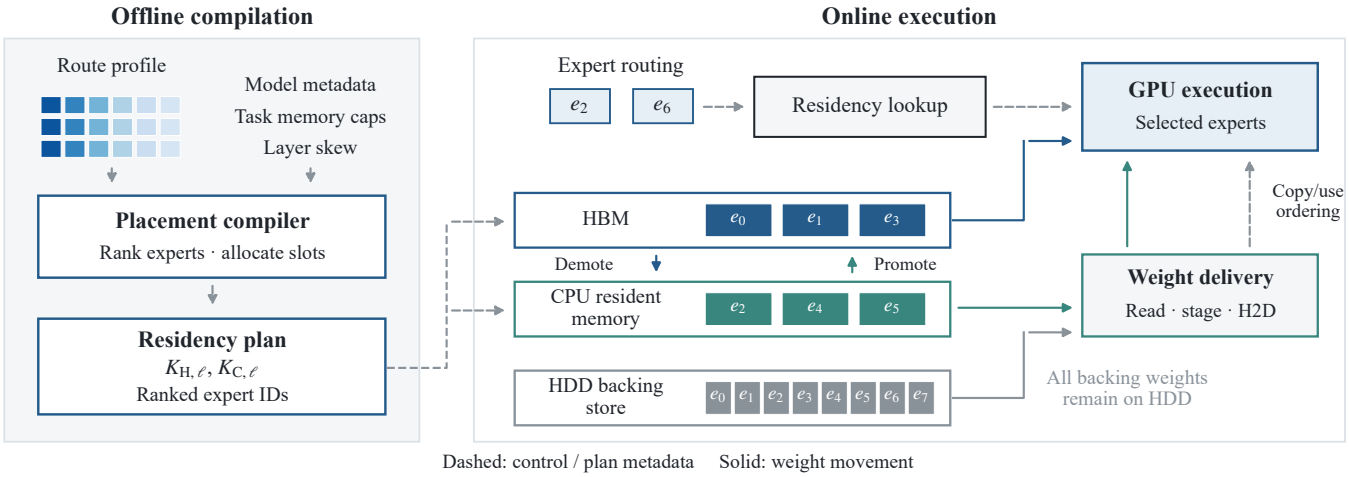


Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution maps routed expert requests to HBM, CPU resident memory, or HDD, then supplies their weights to GPU computation. Dashed arrows carry control or plan metadata; solid arrows move weights.

IV. Overview

Figure 3 follows the plan from profiling to GPU execution. The compiler receives layer/expert counts, skew parameters, model metadata, and expert allowances. It emits capacities and ranked IDs for each layer. Initialization loads the corresponding HBM and CPU residents; the complete expert pool remains on HDD. The runtime subsequently consumes this plan as an initial cache state.

Consider the selected experts e_2 and e_6 in the framework. The residency lookup finds e_2 on the host and e_6 in the backing store. Promoting e_2 frees a host position for an HBM victim. Loading e_6 then admits another GPU resident and updates the same tier order. The delivery path resolves each weight's read and copy before its consumer executes. The model combines the expert outputs using the gate weights and continues to the next layer.

Residency persists across tokens. Each layer updates its own membership within the compiled capacities. Per-layer recency carries request history from one token to the next. Promotions refresh the active experts, and demotions extend their reuse through host residency. Section V specifies these transitions; Section VI examines the resulting delivery cost.

V. Method

A. Compiling Capacities and Initial Identities

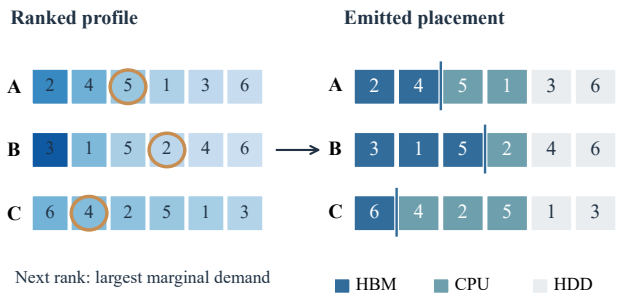


Fig. 4. Compiling the initial state. The left grid contains ranked per-layer demand; circles mark the next uncovered candidates. Shared HBM/CPU slot allowances produce different cuts in the emitted rows. A, B, and C identify different layers.

Consider L MoE layers with N_ℓ experts per layer. Within each model, experts have equal size s and layers use the same top-k.

Expert allowances B_H and B_C provide $S_H = \lfloor B_H/s \rfloor$ HBM slots and $S_C = \lfloor B_C/s \rfloor$ CPU slots.

For each layer, the compiler receives an expert order and a skew parameter α_ℓ. Profile counts determine the order. A discrete Zipf model gives the probability associated with each rank:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{\sum_{j=1}^{N_\ell} j^{-\alpha_\ell}}, \qquad p_{\ell,r} = q_{\ell,r}/L. \tag{2}$$

Here $q_{\ell,r}$ is conditional on an access to layer ℓ. Equal top-k gives equal selection mass across layers. Each rank stores the layer-local ID of its expert weight block.

Figure 4 shows the HBM and CPU cuts in those lists. Capacities satisfy $\sum_\ell K_{H,\ell} \le S_H$, $\sum_\ell K_{C,\ell} \le S_C$, and $K_{H,\ell} + K_{C,\ell} \le N_\ell$. The adaptive scan reserves one slot per layer in each fast tier, then distributes the remaining slots by marginal coverage.

The scan first assigns HBM slots. At each step, it chooses the layer with the largest next-rank score:

$$\Delta_\ell = q_{\ell,K_{H,\ell}+1}. \tag{3}$$

CPU allocation continues from the first ranks beyond each HBM cut. Every allocation exposes a new next-rank candidate in that layer. Layer ID breaks score ties, and expert ID breaks count ties. The direct scan takes $O(L(S_H + S_C))$ time after sorting. Its score is the request probability covered by the next slot.

The emitted plan contains the final capacities and ordered expert IDs. Initialization places each layer's first HBM ranks on the GPU and its next CPU ranks on the host. Natural ID order initializes the uniform control. During execution, admission changes membership within the compiled capacities.

B. Retaining Victims across the Tier Boundary

Each layer maintains two ordered sets, from most-recently used (MRU) to least-recently used (LRU). Their concatenation places HBM residents before CPU residents. An HBM hit moves its expert to HBM MRU. A CPU hit removes its host entry and promotes it to HBM MRU. The GPU LRU victim then becomes CPU MRU. A cold admission follows the same GPU insertion and can remove the CPU LRU. HDD retains the backing weights for every expert.

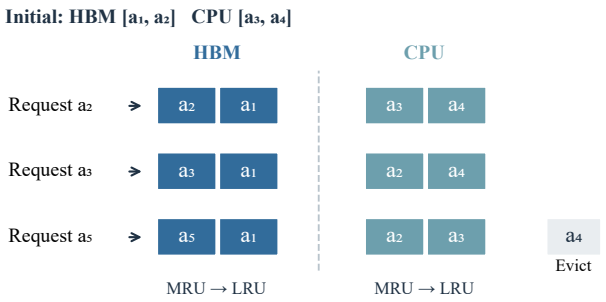


Fig. 5. Residency transitions for three requests. An HBM hit refreshes recency; a CPU hit promotes the requested expert and demotes a GPU victim; cold admission also removes a CPU victim. Rows give the resulting MRU-to-LRU order.

Each row in Figure 5 starts from HBM $[a_1, a_2]$ and CPU $[a_3, a_4]$. An a_2 hit changes the GPU order. Promoting a_3 releases its CPU position, which receives the HBM victim a_2. A cold a_5 request first enters HBM and pushes its GPU victim into the full host cache. The host LRU a_4 then leaves the resident set. Promotion recycles the requested expert's CPU position; cold admission obtains a position through host eviction.

The resulting order extends from HBM MRU through CPU LRU. Requests enter at its front, and capacity pressure removes experts at its tail. During batch preparation, the loader can prepare several active experts. Temporary references hold their weights until each expert processes its token rows.

Each GPU resident carries a host mirror. Demotion admits this mirror to the CPU cache through a metadata update, saving a device-to-host copy. Tracking the consumer stream keeps tensor storage live until kernel execution completes.

C. Ordering Delivery before Expert Consumption

Prefill groups token rows by selected expert and submits the corresponding cold reads. Each read is resolved when its expert reaches consumption. Figure 6 shows resident work overlapping a pending read for another selected expert. Decode delivers each requested expert on demand. The expert kernel processes its assigned rows, and the gate weights combine the resulting outputs.

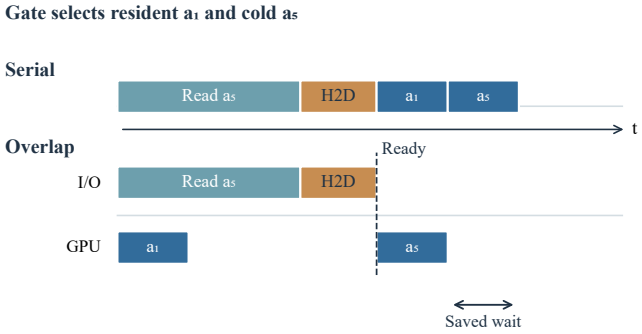


Fig. 6. Prefill delivery for resident a_1 and cold a_5. Computing a_1 overlaps the pending read for a_5. H2D completion releases a_5 for GPU execution; the shortened schedule saves exposed waiting time.

A read future returns the host tensors that supply H2D. The copy stream moves them to the GPU and synchronizes before the loader yields the device tensors. The expert kernel then consumes those tensors. Recording its stream extends their lifetime through kernel completion. This read–copy–compute chain connects storage delivery to the expert's execution point.

Prefetchers use profiles or lookahead to anticipate expert demand [39]–[41]. Batch and request schedulers expose work from concurrent inputs [42], [43]; distributed edge schemes add network delivery [44]. MECC schedules the expert work already selected at the current gate. Retention removes repeated reads and copies. Overlap shortens the exposed part of the remaining delivery.

D. Resolving Requests to Expert Execution

The execution interface accepts a layer/expert ID and its assigned token rows. Lookup resolves the ID to resident tensors or a backing-store block. Delivery prepares GPU weights for the expert projections. Gate weights combine their outputs before execution advances to the next layer.

VI. Evaluation

A. Evaluation Overview

We evaluate decode and first-token speedup, source-tier demand, and workload sensitivity. Phase speedup divides the streaming latency by the policy latency. The main comparison includes SwapMoE-style retention and Fiddler-style CPU computation [9], [12]. Component studies examine allocation, victim retention, and delivery scheduling. Workload sweeps vary calibration, task transfer, input length, and batching. Appendix A provides configuration details and the complete measured policy results.

B. Single-GPU Performance

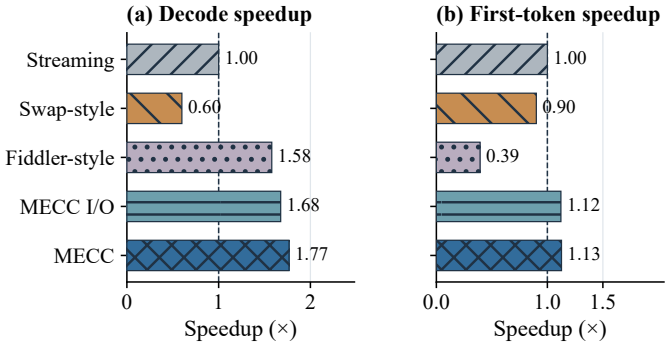


Fig. 7. Decode and first-token speedup on one A100, relative to streaming (1.0). Swap-style and Fiddler-style denote the corresponding mechanism adapters. MECC I/O varies the weight-delivery path. Higher is better.

Figure 7 reports 1.77× decode and 1.13× first-token speedup for MECC over streaming. Against Fiddler-style CPU computation, the gains are 1.12× and 2.86×, respectively. GPU execution reuses each fetched expert across its selected prompt rows. This reuse has more work to cover during prefill, accounting for the different phase tradeoffs. Response length determines how these initial and recurring costs contribute to completion time.

MECC serves 77.5% of classified expert accesses from HBM and 10.0% from the host cache. The remaining 12.5% reach HDD. Each host hit replaces a storage read with an H2D copy; each HBM hit directly supplies GPU computation. Promoting a host resident creates room for the displaced GPU expert, extending its retention into the next tier. Later requests can reuse this host copy after its GPU slot has been reassigned. The access distribution captures the resulting use of the three delivery paths.

Joint retention gives 2.95× decode speedup over the SwapMoE-style adapter. Against independent replacement, the gain is 4.46×, accompanied by 73.9% lower HDD demand in Figure 1. These results link cross-tier retention to the cost of repeated weight delivery. Component studies next examine allocation, retention, and scheduling.

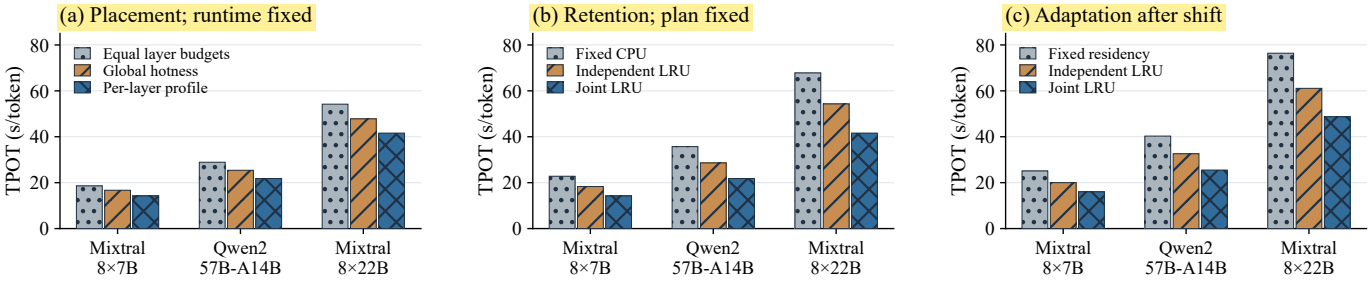


Fig. 8. Component comparisons on three models. (a) Placement with runtime fixed. (b) Retention with initialization fixed. (c) Adaptation after a task switch.

C. Mechanism Attribution

Figure 8 varies one decision in each panel. The placement comparison keeps the runtime rule fixed; retention keeps initialization fixed; adaptation follows a task switch. For Mixtral-8x7B, per-layer placement changes TPOT from 18.6 to 14.2 s/token. Joint retention changes it from 18.3 to 14.2, and online adaptation changes the post-switch value from 25.1 to 16.0. The same comparisons span three models, relating each mechanism to its allocation or request sequence.

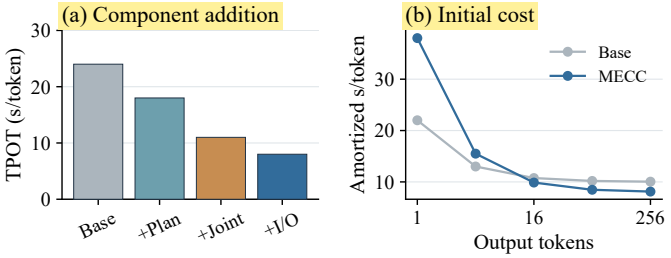


Fig. 9. Ablation and startup. The right panel reports initialization cost per output token plus decode cost.

Adding placement, joint retention, and I/O scheduling gives the TPOT sequence 24, 18, 11, and 8 s/token in Figure 9(a). The successive reductions are 6, 7, and 3 s/token. Joint retention provides the largest step by keeping GPU victims available on the host. Scheduling then shortens the delivery path at 20 percent HDD demand. Panel (b) includes the initial placement cost, connecting this decode gain to the response length.

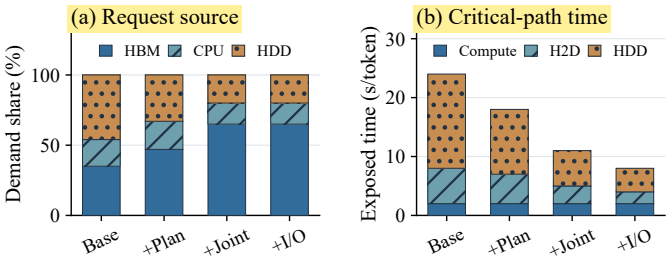


Fig. 10. Source demand and exposed time. The I/O step shortens exposed read/copy time at the final request-source distribution.

Figure 10 separates changes in demand from changes in exposed time. Placement and joint retention lower HDD demand from 46 to 20 percent. The final I/O configuration has an HBM/CPU/HDD mix of 65/15/20 percent. Its TPOT reduction from 11 to 8 s/token comprises 2 seconds less exposed HDD time and 1 second less exposed H2D time. The compute component remains at 2 seconds. Scheduling brings transfer completion closer to ready computation. Retention and overlap therefore act on two parts of delivery cost: the number of transfers and their exposed duration.

D. Model and Deployment Scaling

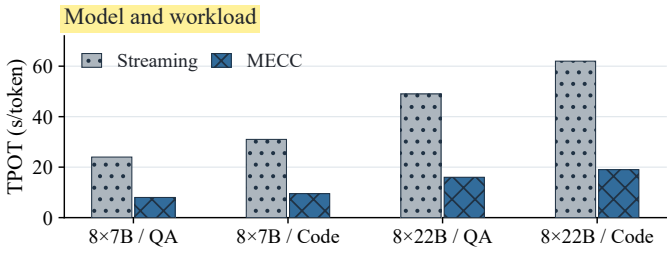


Fig. 11. Model and workload comparison. Mixtral-8x7B and 8x22B with QA/code inputs. Each policy uses the same model and workload within a group.

In Figure 11, MECC's speedup over streaming spans 3.00–3.26× across the four model/workload groups. The absolute TPOT difference grows from 16 seconds on 8x7B QA to 43 seconds on 8x22B Code. Moving from 8x7B to 8x22B doubles MECC TPOT; the corresponding streaming increase is 2.00–2.04×. The larger model therefore saves more delivery time per token at a comparable speedup.

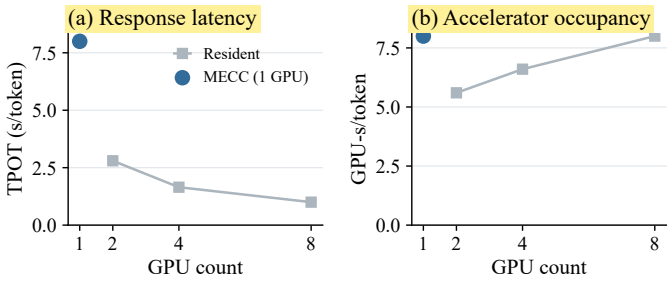


Fig. 12. One-GPU/resident-reference tradeoff. The panels compare single-GPU MECC with resident configurations and report latency alongside GPU occupancy.

The endpoints in Figure 12 pair one GPU at 8 s/token with eight GPUs at 1 s/token. Both use 8 GPU-s/token, with different response times. The two-GPU resident point uses 5.6 GPU-s/token. Plotting the intermediate configurations shows how accelerator provisioning changes both latency and occupancy. The single-GPU point quantifies the latency of supplying weights through the memory hierarchy.

E. Capacity and Workload Sensitivity

MECC gives 1.42× and 5.17× decode speedup over streaming at 30 and 60 GB. HDD demand falls from 32.2% to zero as resident capacity covers all eight experts per layer.

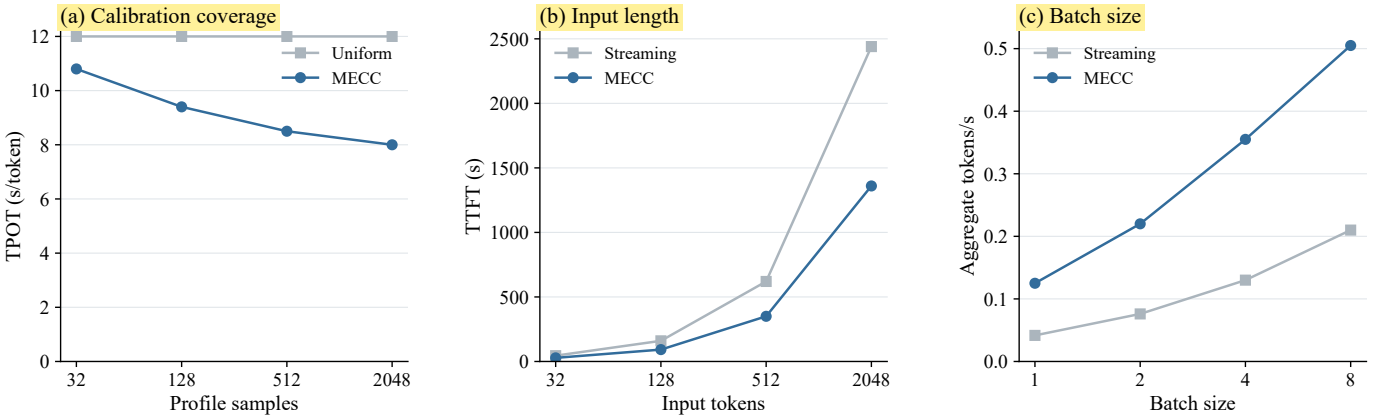


Fig. 13. Profile, input-length, and batch sensitivity. Profile size changes initialization quality, input length changes first-token work, and batch size changes aggregate throughput. The panels vary one workload parameter at a time.

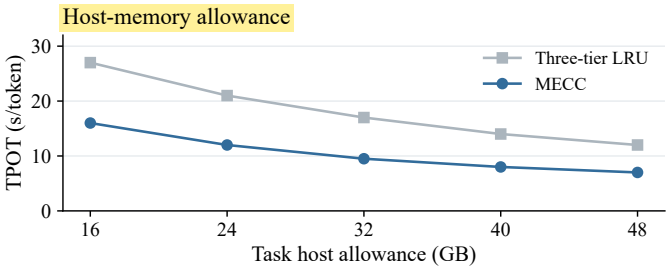


Fig. 14. Decode latency across host-memory allowances.

Figure 14 resolves the capacity range between sparse and broader host retention. MECC changes from 16 to 7 s/token across 16–48 GB; three-tier LRU changes from 27 to 12. The first host increment saves 4 seconds/token for MECC, while the final increment saves 1 second/token. Earlier slots can retain victims with shorter reuse distances. Additional slots then cover less frequent returns. This diminishing latency reduction complements the coverage transition at 60 GB, where the primary resident sets contain every expert.

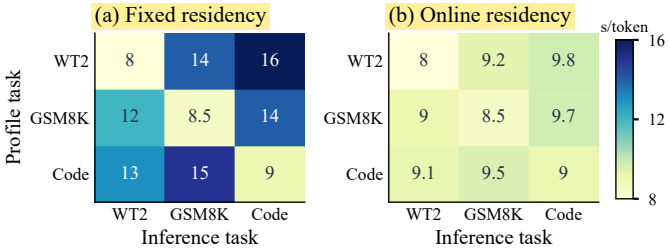


Fig. 15. Profile transfer across tasks. Rows select the calibration task and columns the inference task. Cells are TPOT in s/token; the two panels share a color scale.

Figure 15 separates within-task calibration from transfer to another task. The mean diagonal TPOT is 8.50 s/token under both fixed and online residency. Across the six off-diagonal pairs, the mean changes from 14.00 to 9.38 s/token, a 33.0 percent reduction. Request-driven admission brings the cross-task resident set toward the experts selected by the serving workload. The response to calibration size in Figure 13(a) addresses a different effect: how much profile information is available before the first request.

F. Calibration Coverage and Input Length

Increasing calibration from 32 to 2048 samples changes TPOT from 10.8 to 8.0 s/token in Figure 13(a). The final expansion yields a smaller reduction than the first. Once the ranked hot set stabilizes, additional samples affect fewer initial placements.

Figure 13(b) scales the prompt from 32 to 2048 tokens. MECC's TTFT changes from 28 to 1360 seconds, compared with 45 to 2440 for streaming. Their endpoint speedups are 1.61× and 1.79×. Prompt rows selected for the same expert can share one fetched weight block. Increasing input length thus exposes more opportunities to amortize delivery within prefill, alongside the increase in prompt computation.

G. Batch Throughput and Sequence Progress

Figure 13(c) reports aggregate output rate. Moving from one to eight sequences increases MECC throughput from 0.125 to 0.505 tokens/s, or 4.04×. The additional sequences share fetched expert weights and introduce more active rows at each gate.

For a synchronous batch of B sequences with aggregate rate $R(B)$, the interval between batch steps is $B/R(B)$. The plotted MECC rates give 8.00 seconds at batch one and 15.84 seconds at batch eight. Batching therefore processes more tokens per unit time while increasing the interval between successive tokens of one sequence.

H. Amortizing Initial Placement

Let ΔI denote the extra initialization time and ΔD the per-token decode saving. Equating the amortized per-token times gives the crossing $n^\star = \Delta I/\Delta D$. Figure 9(b) uses 18 seconds of extra initialization and 2 seconds of decode saving, yielding 9 output tokens. Longer responses recover the placement cost and retain part of the decode gain.

At 64 output tokens, initialization contributes 0.47 s/token for MECC and 0.19 for the reference. Their total averages become 8.47 and 10.19 s/token, respectively. When requests share a resident cache, their output tokens amortize the same initialization. The cache lifetime determines the amortization horizon.

VII. Conclusion

MECC combines profile-guided initial placement with joint HBM/CPU retention for single-GPU MoE inference. Its allocation and admission rules reduce repeated weight delivery, while scheduling overlaps the remaining read/copy work with ready expert computation. On one A100, MECC gives 1.77× decode speedup over streaming and 1.12× over the Fiddler-style adapter.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[10] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[11] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[12] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[13] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[14] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[15] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[16] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[17] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[18] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[19] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[20] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[21] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[22] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[23] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[24] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[25] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[26] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[27] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[28] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[29] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[30] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[31] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.

[32] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[33] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[34] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[35] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[36] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[37] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[38] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[39] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[40] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[41] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[42] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[43] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[44] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

Appendix A
Experimental Configuration and Figure Data

A. Hardware and Weight Representation

The testbed uses one NVIDIA A100 80 GB PCIe with FP16 expert weights. Dense layers and the router execute on the GPU. The expert store is a RAID logical volume reporting ROTA=1, called HDD in this paper. The controller's physical-drive inventory remains unverified. Each expert block contains gate, up, and down tensors identified by layer and expert ID. HDD retains the complete expert pool.

B. Measured Protocols

The main comparison uses Mixtral-8x7B-Instruct-v0.1 with uniform initialization, four HBM slots and three CPU slots per layer. Planner allowances are 50/40 GB. Each policy runs once on eight WikiText-2 prompts with 64 input and 64 output tokens. PPL uses a 1024-token WikiText-2 sample. The loader and model graph are shared.

Raw TPOT in seconds is 13.04 for streaming, 21.75 for SwapMoE-style, 8.26 for Fiddler-style, 7.19 for FlashMoE-RF, 7.78 for MECC I/O, and 7.38 for MECC. Independent caching gives 32.94. TTFT is 156.86 seconds for streaming, 139.14 for MECC, 398.45 for Fiddler-style, and 101.76 for FlashMoE-RF. Plotted speedup divides the streaming phase latency by policy latency using unrounded JSON fields. The additional FlashMoE-RF adapter gives 1.81× decode and 1.54× first-token speedup over streaming. These complete policy results supplement the five-policy main figure.

The separate 30/60 GB host comparison uses four WikiText-2 prompts, 16 output tokens, and four HBM slots per layer. At 60 GB, four HBM and four CPU slots cover all eight experts per layer.

C. Policy Definitions

Streaming retains the initialized HBM set and fetches misses without GPU admission. The SwapMoE-style FP16 adapter uses a fixed CPU set and GPU LRU replacement. Fiddler-style computes non-HBM experts on the CPU. Both are local mechanism implementations. Independent caching updates HBM/CPU recency separately; MECC couples GPU promotion to victim demotion. MECC I/O is a separate delivery control.

FlashMoE-RF combines public inverse-recency and frequency features with equal weights, using HBM LRU. Its host score is $0.5/d_e + 0.5 f_e / f_{\max}$, with request distance d_e and count f_e. The lowest score is evicted; ID breaks ties. The original trained predictor is absent. The main figure abbreviates SwapMoE-style as Swap-style. MoE-CORE and MoE-Infinity have no completed run in this repository.

D. Timing and Demand Counters

TTFT is time to first token; TPOT is time per output token during decode. Accelerator occupancy is GPU count times TPOT, in GPU-s/token. Measured HDD-demand shares aggregate classified accesses across instrumented phases. Prefetched experts are classified by residency at consumption. These shares are separate from physical read/copy byte counters and decode-only demand. The short-window diagnostic compares streaming and MECC routing, logits, gates, and tokens through their shared dispatch; it is not independent full-model equivalence evidence.

E. Figure Data Annotations

Yellow-highlighted values and panel titles identify authored design targets: unmeasured values specifying the illustrated comparisons. Introduction and main-runtime plots use committed A100 measurements. Motivation panel (b) uses measured aggregate demand; panels (a,c) use design arrays. Panels (d–f), the framework, and method figures are mechanism examples.

Component, startup, traffic/exposure, cross-model, GPU-occupancy, quota, sensitivity, and profile-transfer plots use design targets. Model names, QA/Code groups, and WikiText-2/GSM8K labels specify intended comparisons, not additional completed benchmark runs. Startup uses $T_{\text{avg}}(n) = T_{\text{init}}/n + T_{\text{decode}}$. Demand stacks sum to 100%; exposed-time stacks sum to component TPOT. Derived differences, speedups, off-diagonal means, and batch-step intervals use the same unmeasured arrays. They are arithmetic summaries, not additional runs.

F. Memory and Execution Accounting

Planner allowances count primary slots. GPU host mirrors and staging add physical host allocations; equal slot counts do not establish equal peak host bytes. The compiler library accepts per-layer skew and ranked IDs. The non-uniform e2e CLI generates skew; calibration is a separate compiler input. The measured main run uses uniform initialization. Prefill submits selected-batch read futures; decode uses demand delivery. Copy-stream synchronization orders CUDA consumption, and consuming-stream tracking preserves tensor lifetime.

G. Source Records

Committed run JSONs supply measurements. The measured-figure and ratio audits record run IDs, fields, raw latencies, and SHA-256 hashes. The two design-array files supply highlighted figures and retain their source hashes. Each archived manuscript includes its source and artifact hash record.